

LABORATORY WORK 4

SQL Queries with the Airline Database

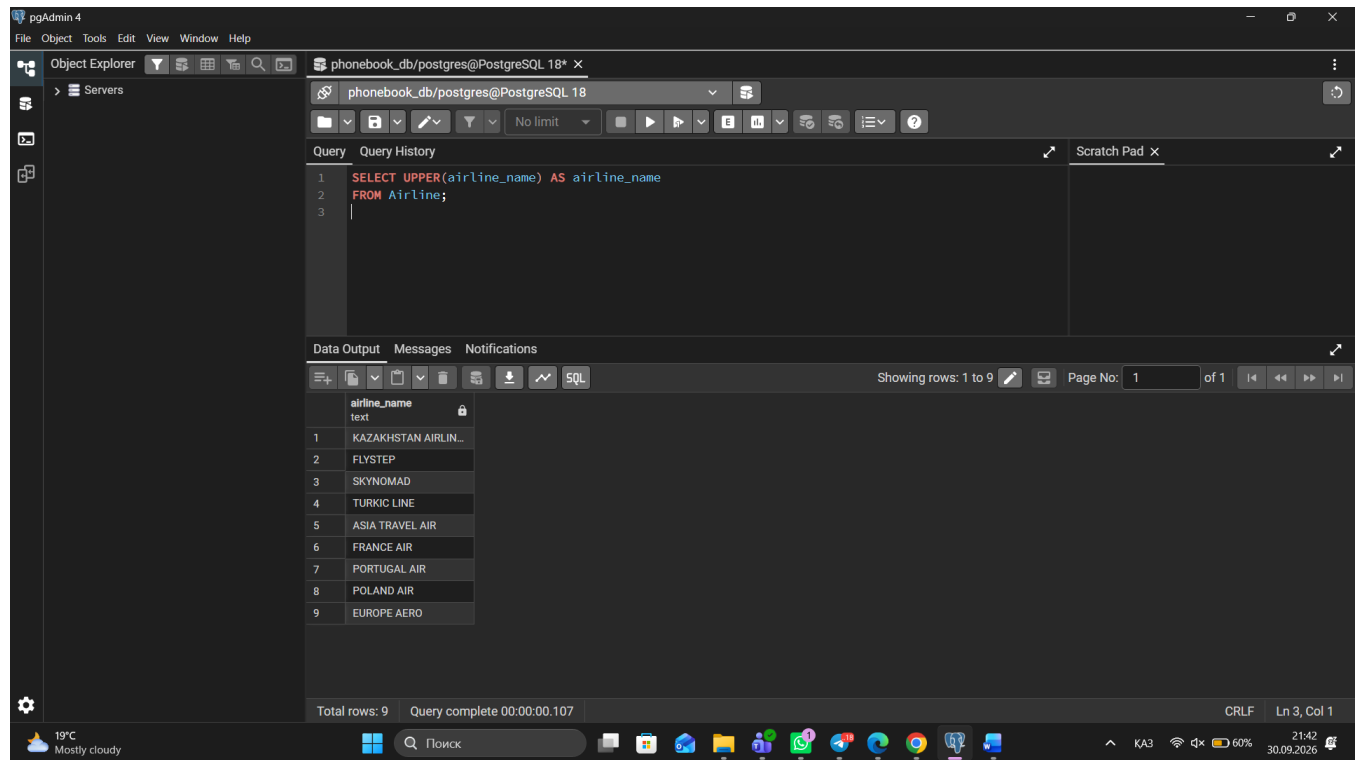
Introduction

In this laboratory work, I use the airline database from the previous laboratory works. The database was prepared so that all 15 tasks have data to return. The SQL queries use simple PostgreSQL functions and conditions.

Task 1. Retrieve all airline names in uppercase.

Explanation: UPPER changes all letters to uppercase.

```
SELECT UPPER(airline_name) AS airline_name
FROM Airline;
```



The screenshot shows the pgAdmin 4 interface. The query editor displays the following SQL query:

```
1 SELECT UPPER(airline_name) AS airline_name
2 FROM Airline;
3
```

The Data Output pane shows the results of the query, displaying 9 rows of airline names in uppercase:

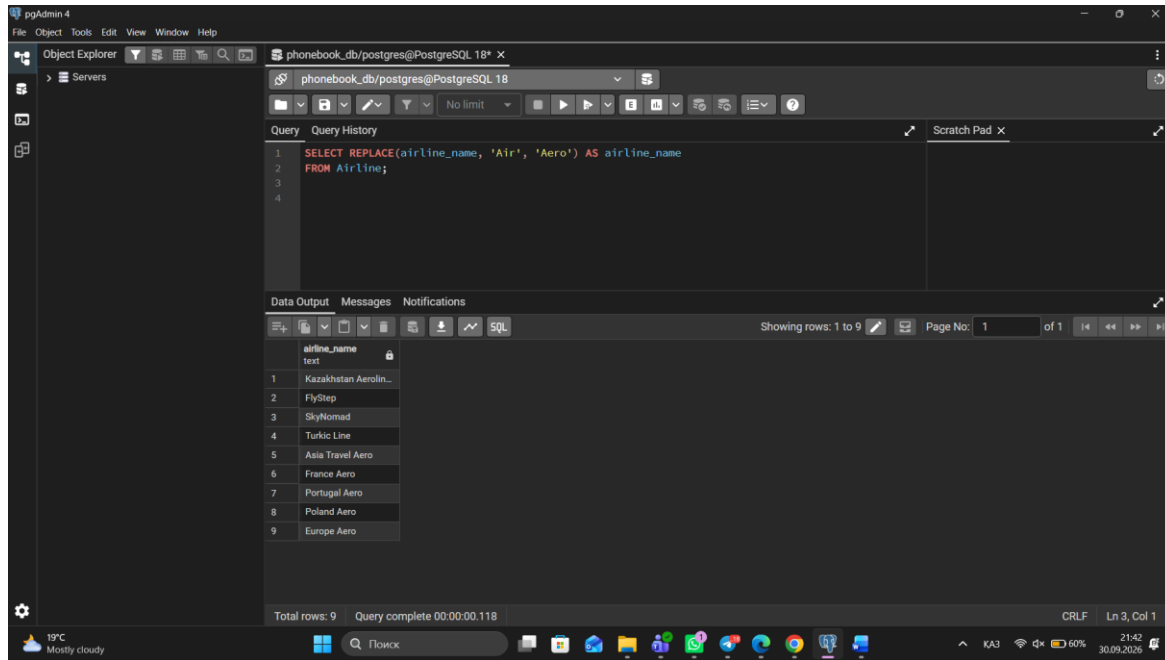
airline_name
KAZAKHSTAN AIRLIN...
FLYSTEP
SKYNOMAD
TURKIC LINE
ASIA TRAVEL AIR
FRANCE AIR
PORTUGAL AIR
POLAND AIR
EUROPE AERO

The status bar at the bottom indicates "Total rows: 9" and "Query complete 00:00:00.107". The system tray at the bottom shows the date and time as 21:42 on 30.09.2026.

Task 2. Replace any occurrence of the word "Air" with "Aero".

Explanation: REPLACE changes the word Air to Aero.

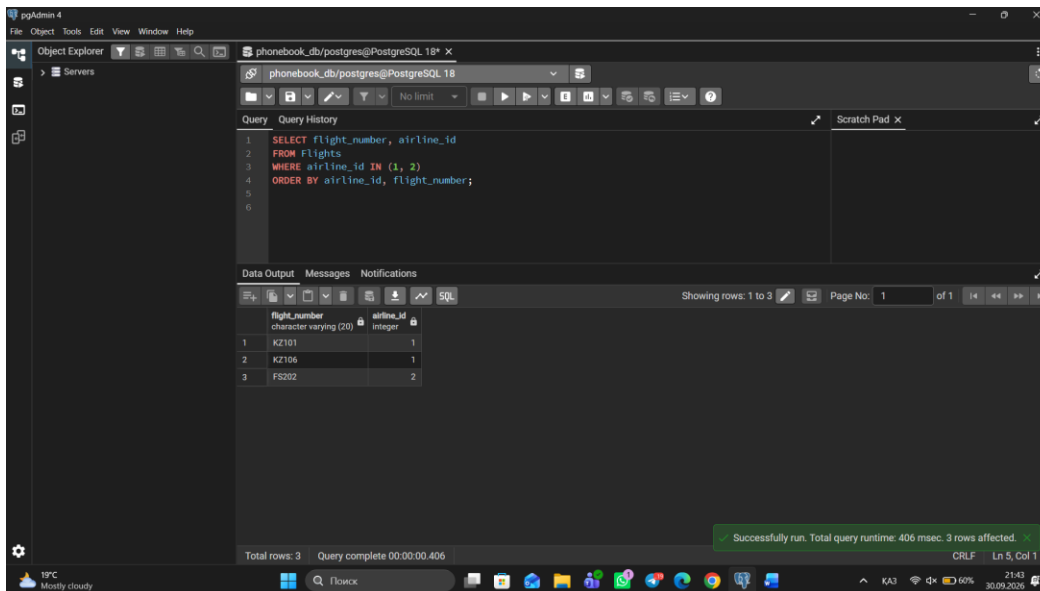
```
SELECT REPLACE(airline_name, 'Air', 'Aero') AS airline_name
FROM Airline;
```



Task 3. Find flight numbers connected with airline 1 and airline 2.

Explanation: IN (1, 2) selects flights that belong to airline 1 or airline 2.

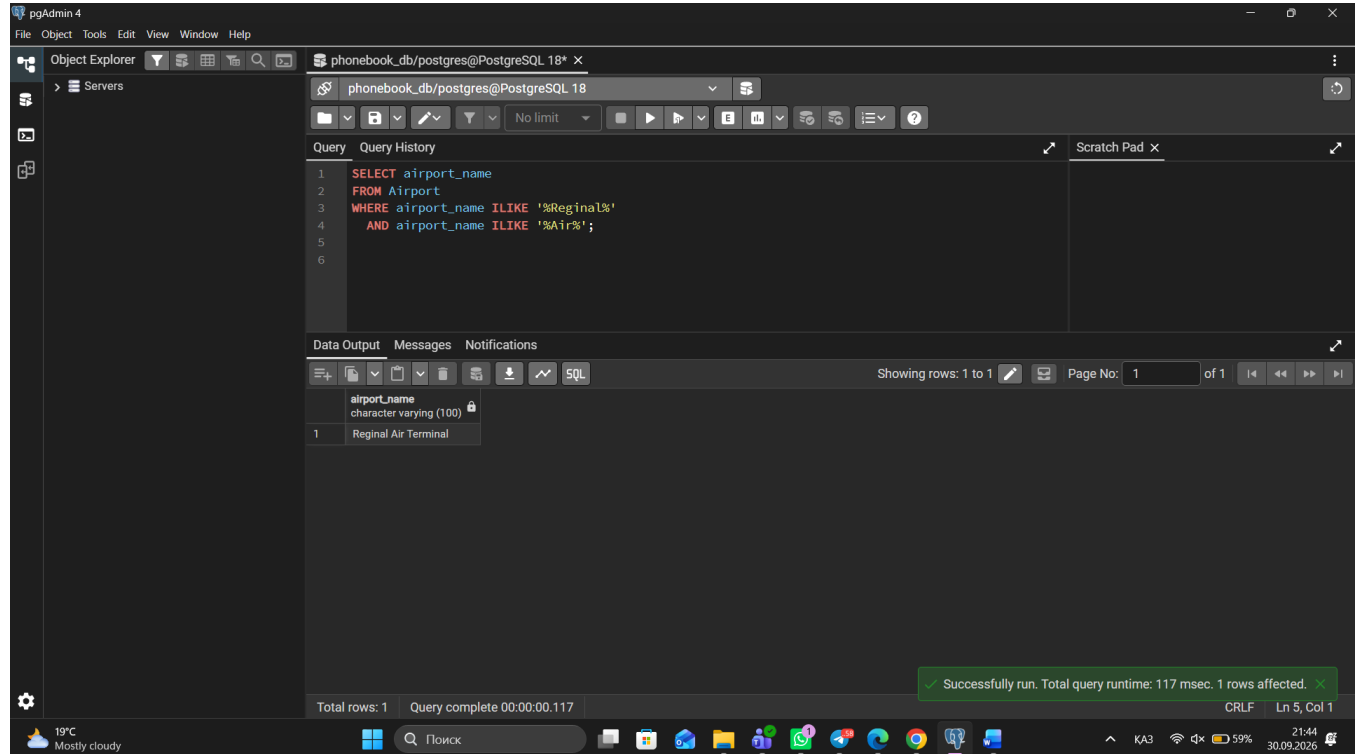
```
SELECT flight_number, airline_id
FROM Flights
WHERE airline_id IN (1, 2)
ORDER BY airline_id, flight_number;
```



Task 4. Retrieve airports that contain the words "Reginal" and "Air".

Explanation: ILIKE searches for both words without case sensitivity.

```
SELECT airport_name
FROM Airport
WHERE airport_name ILIKE '%Reginal%'
      AND airport_name ILIKE '%Air%';
```



The screenshot shows the pgAdmin 4 application window. The left sidebar displays the 'Object Explorer' with a tree view showing 'Servers' and 'phonebook_db/postgres@PostgreSQL 18'. The main pane is titled 'phonebook_db/postgres@PostgreSQL 18' and contains a SQL query editor. The query is as follows:

```
1 SELECT airport_name
2 FROM Airport
3 WHERE airport_name ILIKE '%Reginal%'
4 AND airport_name ILIKE '%Air%';
5
6
```

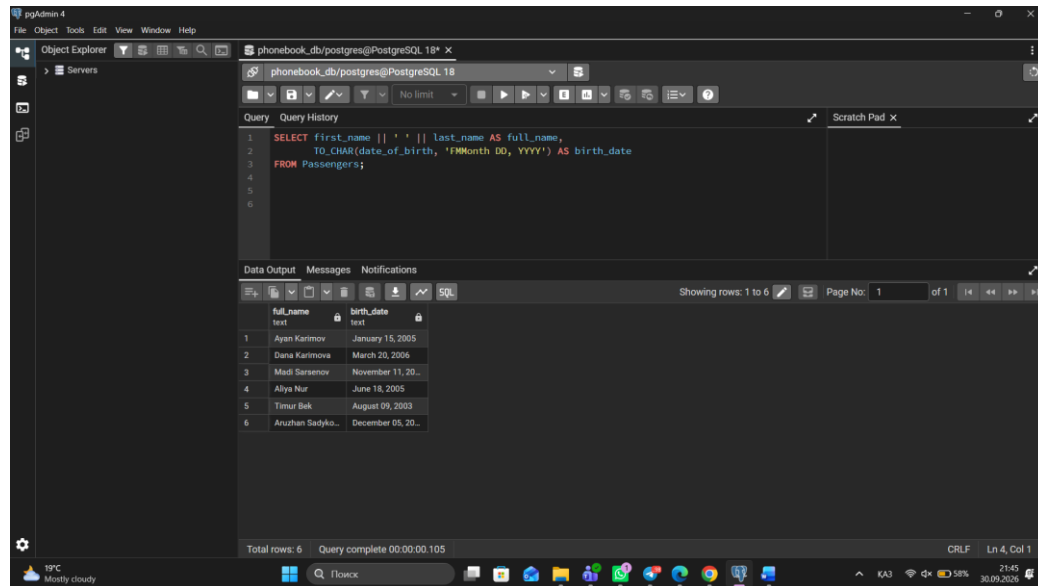
Below the query editor, the 'Data Output' tab is active, showing the results of the query. The results are displayed in a table with one column, 'airport_name', and one row, 'Reginal Air Terminal'. The table is titled 'airport_name character varying (100)'.

At the bottom of the window, a status bar indicates 'Total rows: 1' and 'Query complete 00:00:00.117'. A green notification bar at the bottom right states 'Successfully run. Total query runtime: 117 msec. 1 rows affected.'

Task 5. Retrieve passenger names and format birth dates as "Month DD, YYYY".

Explanation: TO_CHAR changes the date into the required format.

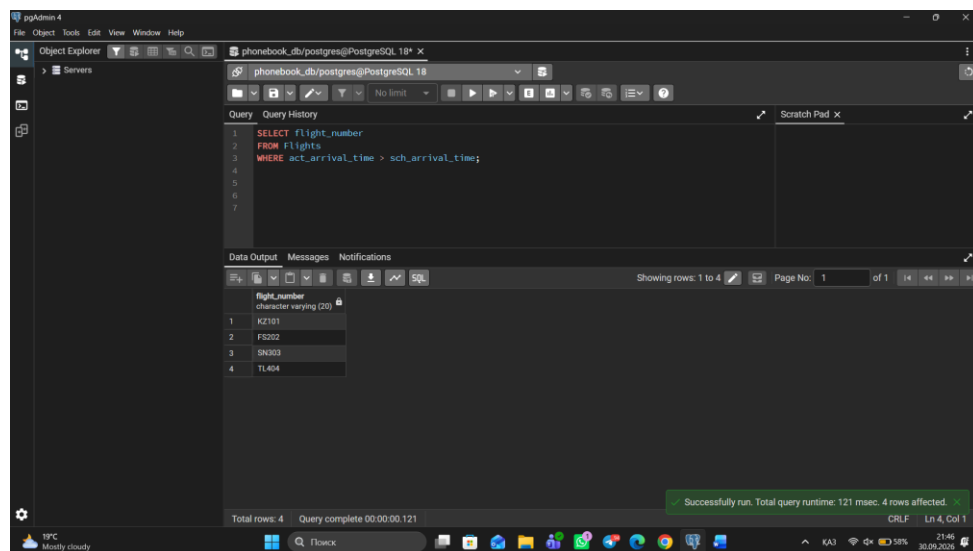
```
SELECT first_name || ' ' || last_name AS full_name,  
       TO_CHAR(date_of_birth, 'FMMonth DD, YYYY') AS birth_date  
FROM Passengers;
```



Task 6. Find flight numbers that have been delayed based on the actual arrival time.

Explanation: The flight is delayed when actual arrival is later than scheduled arrival.

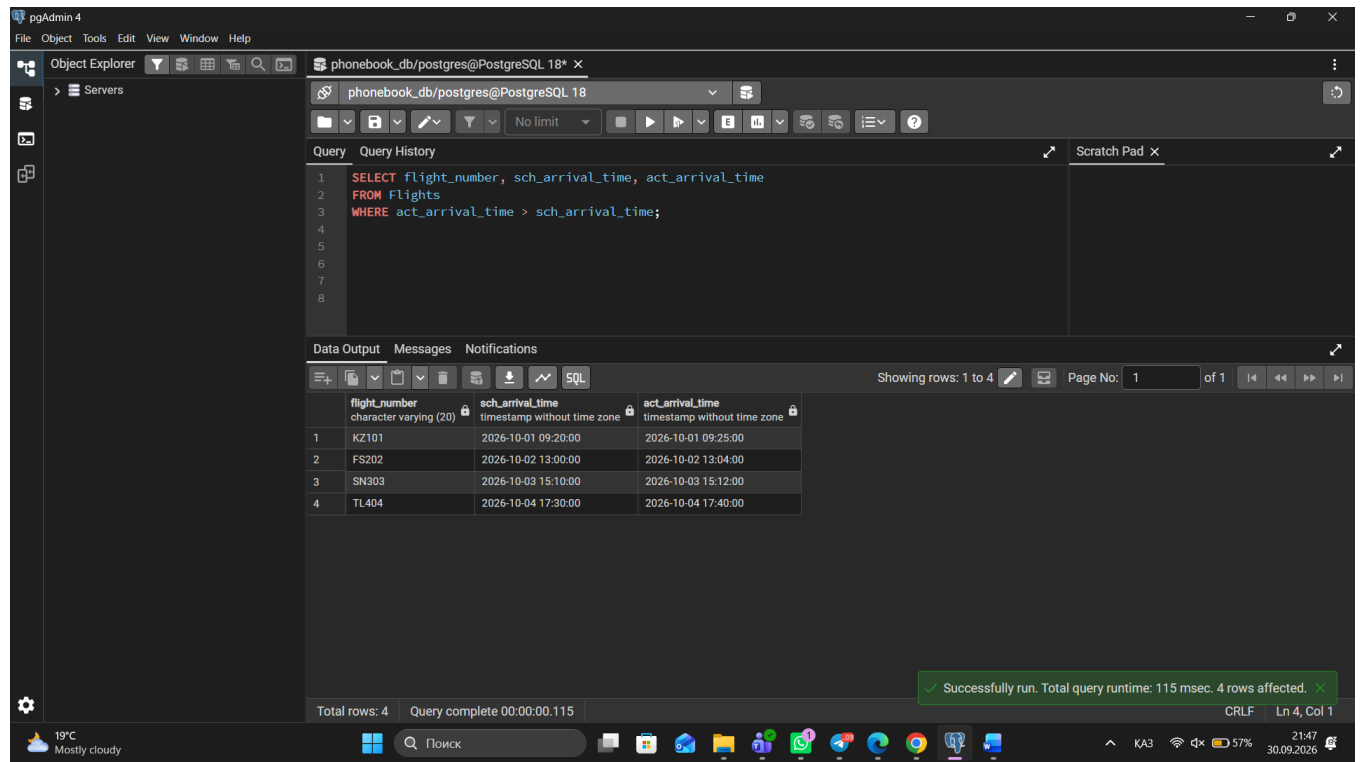
```
SELECT flight_number  
FROM Flights  
WHERE act_arrival_time > sch_arrival_time;
```



Task 7. Find flights that arrived late.

Explanation: The same time comparison shows flights that arrived late.

```
SELECT flight_number, sch_arrival_time, act_arrival_time
FROM Flights
WHERE act_arrival_time > sch_arrival_time;
```



The screenshot shows the pgAdmin 4 interface. The query editor contains the following SQL query:

```
1 SELECT flight_number, sch_arrival_time, act_arrival_time
2 FROM Flights
3 WHERE act_arrival_time > sch_arrival_time;
4
5
6
7
8
```

The Data Output tab shows the results of the query, displaying 4 rows of data:

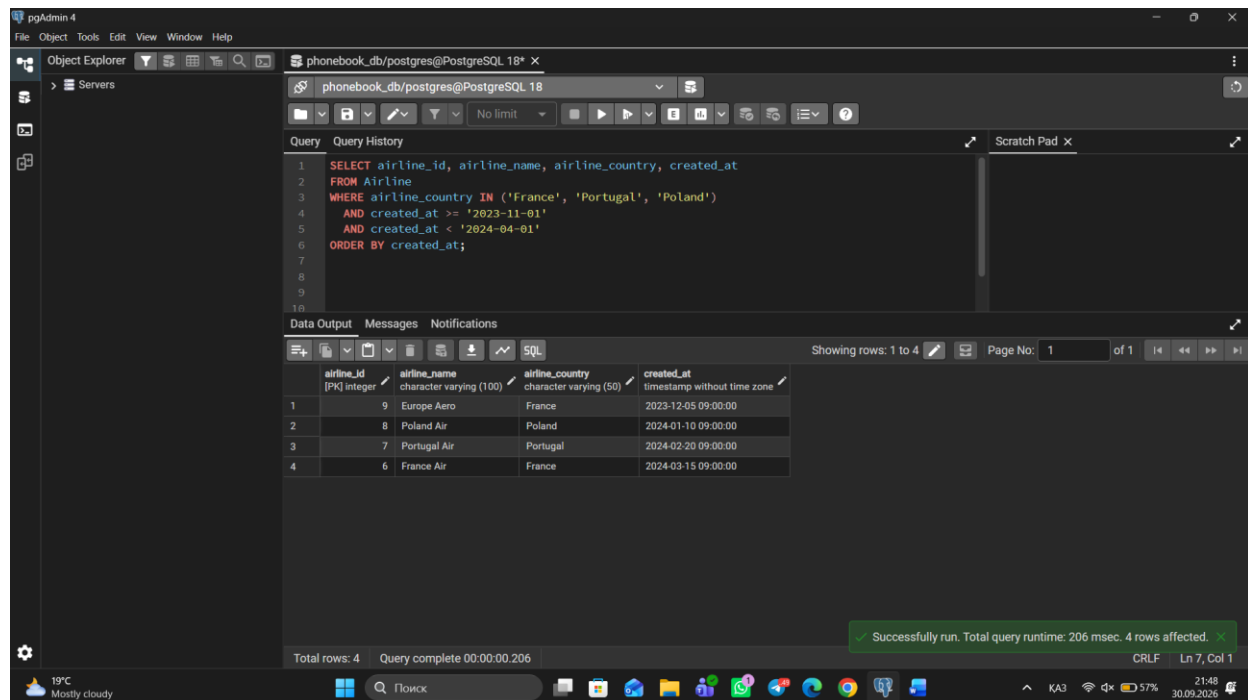
	flight_number character varying (20)	sch_arrival_time timestamp without time zone	act_arrival_time timestamp without time zone
1	KZ101	2026-10-01 09:20:00	2026-10-01 09:25:00
2	FS202	2026-10-02 13:00:00	2026-10-02 13:04:00
3	SN303	2026-10-03 15:10:00	2026-10-03 15:12:00
4	TL404	2026-10-04 17:30:00	2026-10-04 17:40:00

At the bottom of the interface, a green status bar indicates: "Successfully run. Total query runtime: 115 msec. 4 rows affected." The bottom of the screen shows a Windows taskbar with the date 30.09.2026 and time 21:47.

Task 8. Show airlines from France, Portugal, or Poland created between November 1, 2023 and March 31, 2024.

Explanation: IN checks the countries and the date conditions select the required period.

```
SELECT airline_id, airline_name, airline_country, created_at
FROM Airline
WHERE airline_country IN ('France', 'Portugal', 'Poland')
      AND created_at >= '2023-11-01'
      AND created_at < '2024-04-01'
ORDER BY created_at;
```



The screenshot shows the pgAdmin 4 interface. The query editor displays the following SQL query:

```
1 SELECT airline_id, airline_name, airline_country, created_at
2 FROM Airline
3 WHERE airline_country IN ('France', 'Portugal', 'Poland')
4      AND created_at >= '2023-11-01'
5      AND created_at < '2024-04-01'
6 ORDER BY created_at;
```

The Data Output tab shows the results of the query, displaying 4 rows. The columns are: airline_id, airline_name, airline_country, and created_at.

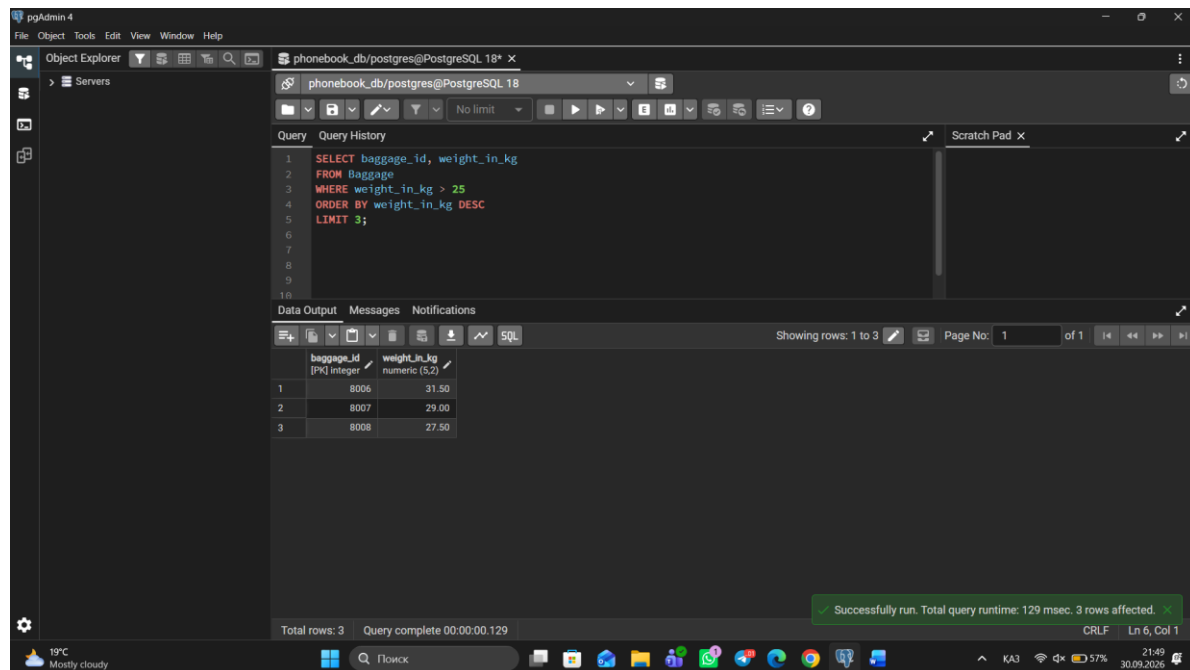
airline_id	airline_name	airline_country	created_at
9	Europe Aero	France	2023-12-05 09:00:00
8	Poland Air	Poland	2024-01-10 09:00:00
7	Portugal Air	Portugal	2024-02-20 09:00:00
6	France Air	France	2024-03-15 09:00:00

At the bottom, a green status bar indicates: "Successfully run. Total query runtime: 206 msec. 4 rows affected."

Task 9. Find the top 3 overweighted baggage items heavier than 25 kg.

Explanation: The query filters weights over 25 kg and shows the three largest weights.

```
SELECT baggage_id, weight_in_kg
FROM Baggage
WHERE weight_in_kg > 25
ORDER BY weight_in_kg DESC
LIMIT 3;
```



The screenshot shows the pgAdmin 4 interface with a SQL query executed in the 'Query' tab. The query is:

```
1 SELECT baggage_id, weight_in_kg
2 FROM Baggage
3 WHERE weight_in_kg > 25
4 ORDER BY weight_in_kg DESC
5 LIMIT 3;
```

The 'Data Output' tab shows the results of the query in a table with two columns: 'baggage_id' (integer) and 'weight_in_kg' (numeric). The results are:

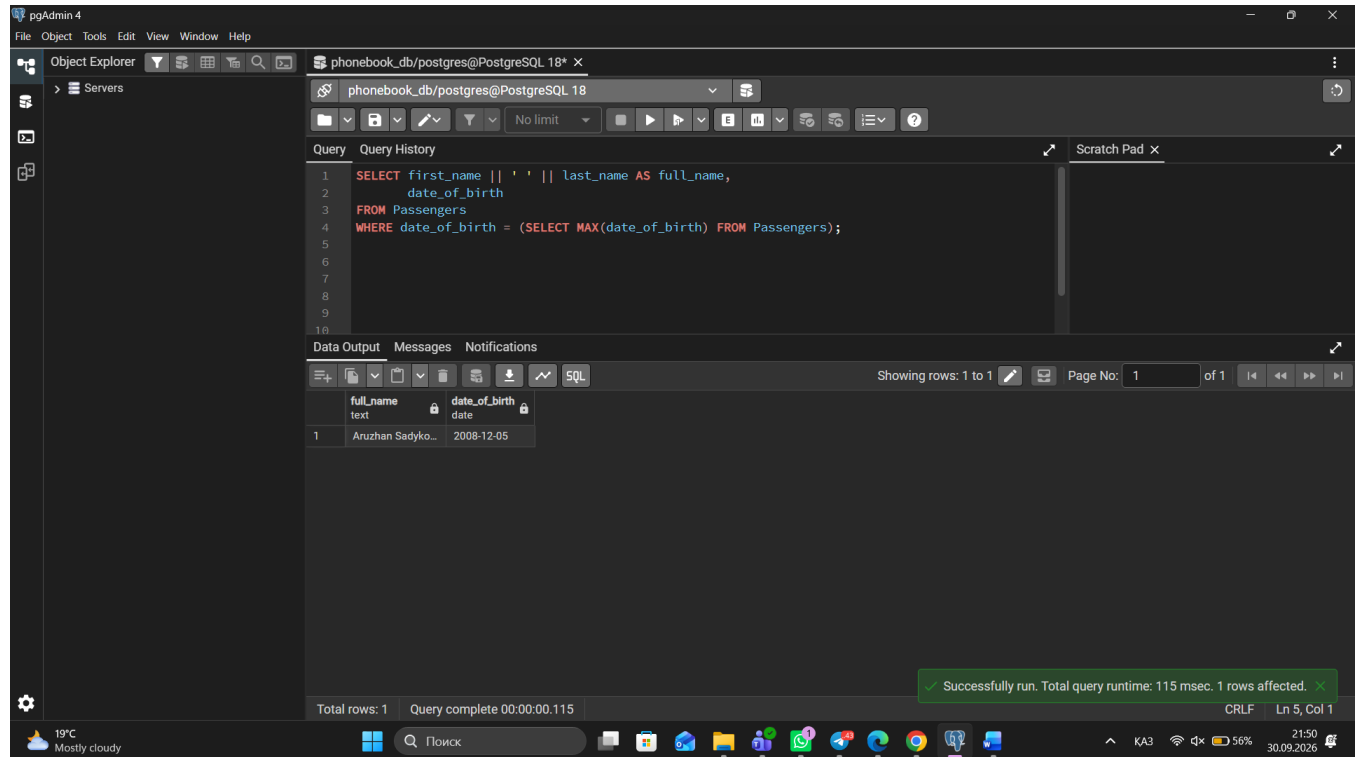
baggage_id	weight_in_kg
1	31.50
2	29.00
3	27.50

The status bar at the bottom indicates: 'Total rows: 3', 'Query complete 00:00:00.129', and 'Successfully run. Total query runtime: 129 msec. 3 rows affected.'

Task 10. Find the youngest passenger's full name.

Explanation: The latest date of birth is the youngest passenger.

```
SELECT first_name || ' ' || last_name AS full_name,  
       date_of_birth  
FROM Passengers  
WHERE date_of_birth = (SELECT MAX(date_of_birth) FROM Passengers);
```



The screenshot shows the pgAdmin 4 application window. The main pane displays a SQL query that selects the full name and date of birth from the Passengers table, filtering for the youngest passenger (maximum date of birth). The query is as follows:

```
1 SELECT first_name || ' ' || last_name AS full_name,  
2       date_of_birth  
3 FROM Passengers  
4 WHERE date_of_birth = (SELECT MAX(date_of_birth) FROM Passengers);
```

Below the query editor, the Data Output pane shows the results of the query. It displays a single row with the following data:

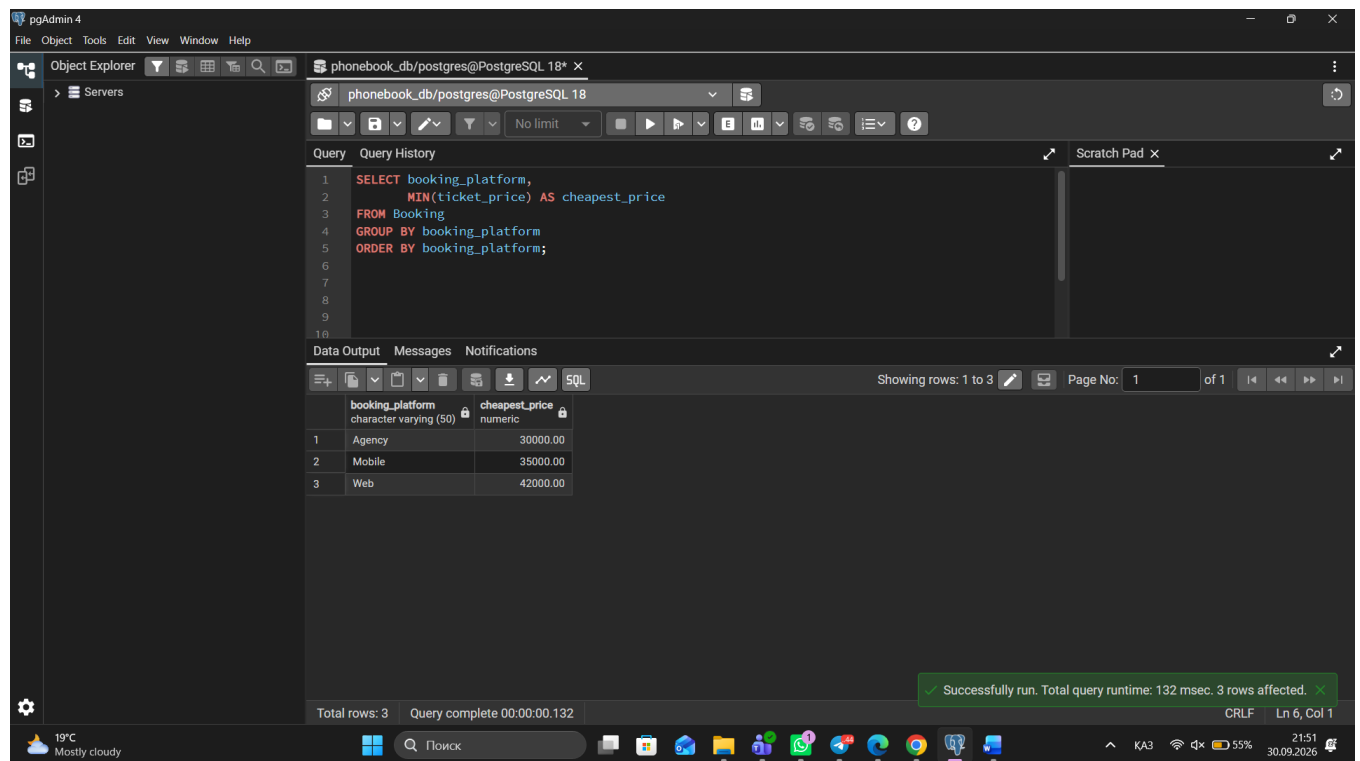
	full_name	date_of_birth
1	Aruzhan Sadyko...	2008-12-05

A green status bar at the bottom of the application indicates: "Successfully run. Total query runtime: 115 msec. 1 rows affected."

Task 11. Find the cheapest booking price on each booking platform.

Explanation: MIN finds the lowest ticket price in each platform.

```
SELECT booking_platform,  
       MIN(ticket_price) AS cheapest_price  
FROM Booking  
GROUP BY booking_platform  
ORDER BY booking_platform;
```



The screenshot shows the pgAdmin 4 interface with a SQL query executed in the 'Query' tab. The query is:

```
1 SELECT booking_platform,  
2        MIN(ticket_price) AS cheapest_price  
3 FROM Booking  
4 GROUP BY booking_platform  
5 ORDER BY booking_platform;
```

The 'Data Output' tab shows the results of the query in a table with 3 rows:

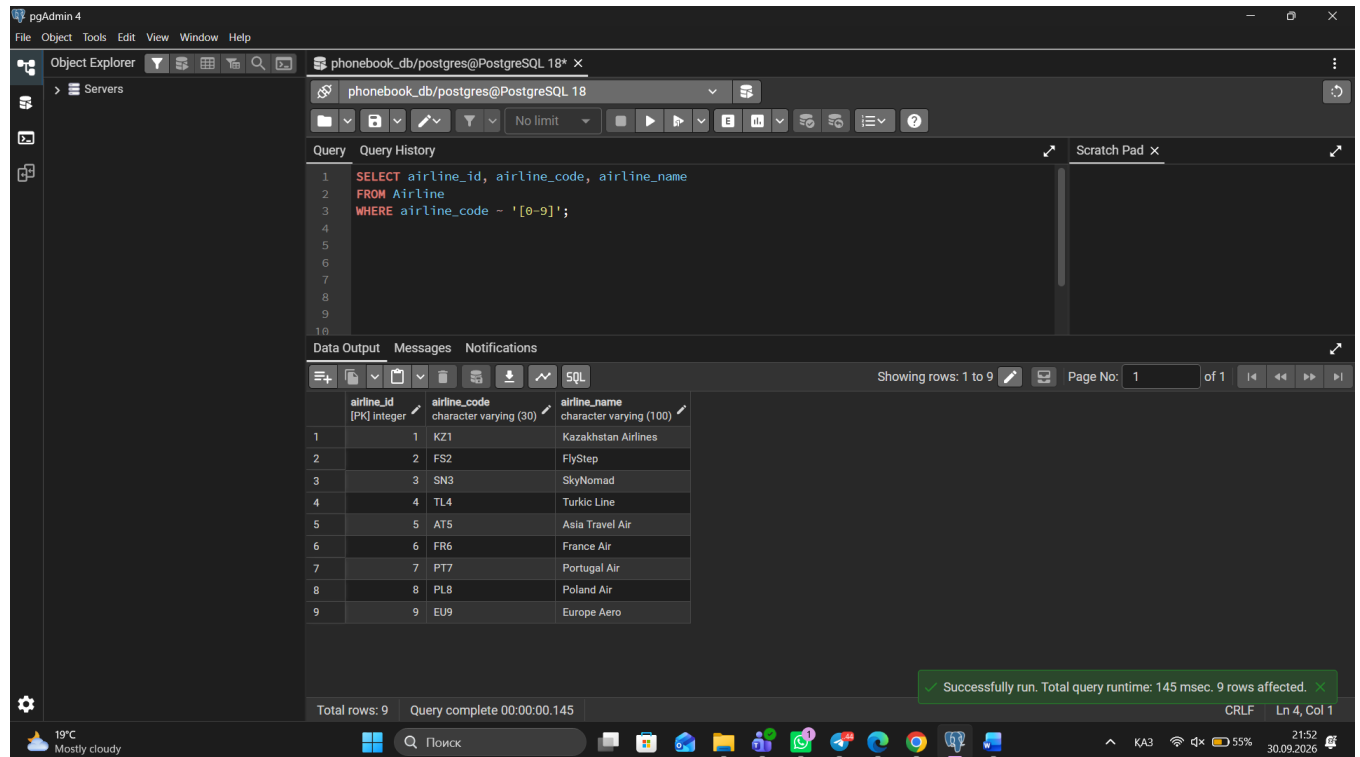
	booking_platform	cheapest_price
1	Agency	30000.00
2	Mobile	35000.00
3	Web	42000.00

A green status bar at the bottom indicates: 'Successfully run. Total query runtime: 132 msec. 3 rows affected.'

Task 12. Return airlines whose airline_code contains a digit.

Explanation: The regular expression `[0-9]` checks for a digit.

```
SELECT airline_id, airline_code, airline_name
FROM Airline
WHERE airline_code ~ '[0-9]';
```



The screenshot shows the pgAdmin 4 interface with a query executed in the 'phonebook_db/postgres@PostgreSQL 18*' database. The query is:

```
1 SELECT airline_id, airline_code, airline_name
2 FROM Airline
3 WHERE airline_code ~ '[0-9]';
```

The results are displayed in the 'Data Output' tab, showing 9 rows. The columns are 'airline_id' (integer), 'airline_code' (character varying (30)), and 'airline_name' (character varying (100)).

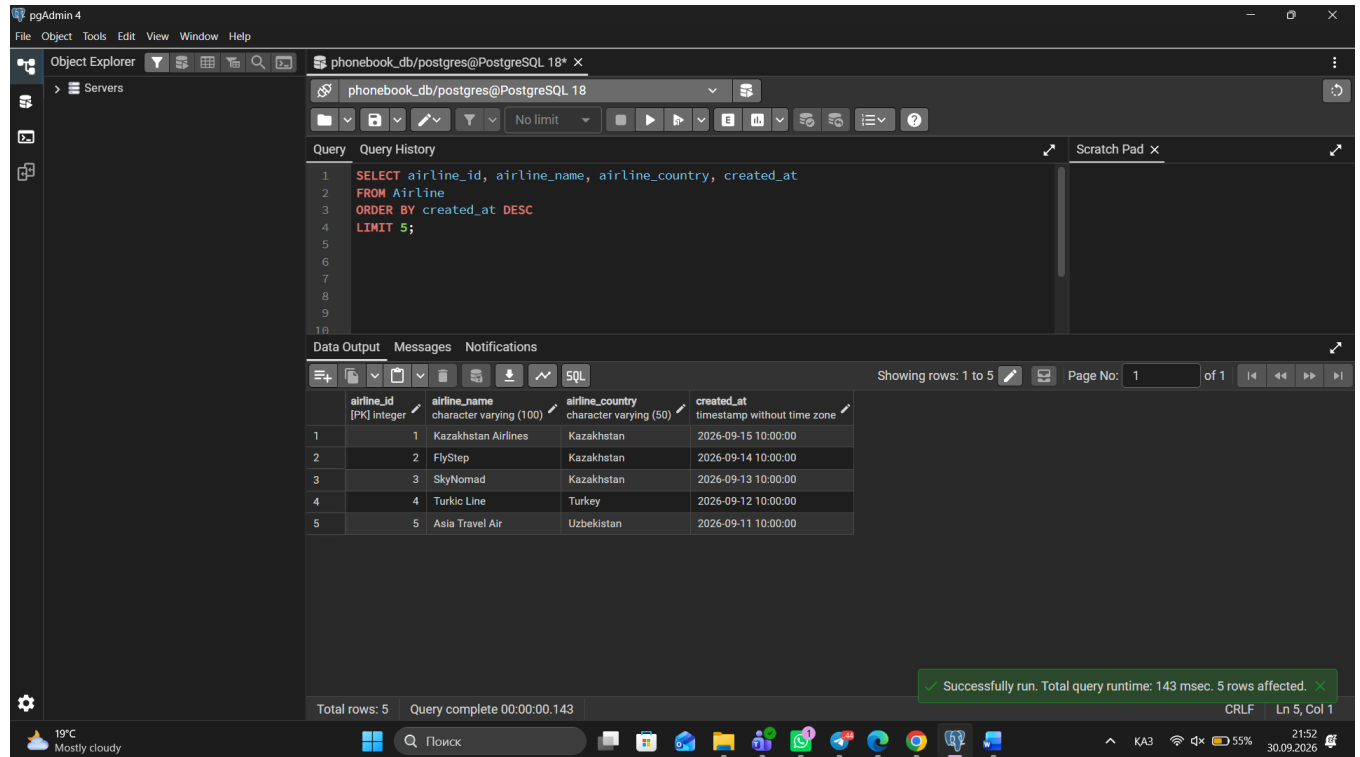
airline_id	airline_code	airline_name
1	KZ1	Kazakhstan Airlines
2	FS2	FlyStep
3	SN3	SkyNomad
4	TL4	Turkic Line
5	AT5	Asia Travel Air
6	FR6	France Air
7	PT7	Portugal Air
8	PL8	Poland Air
9	EU9	Europe Aero

A green status bar at the bottom indicates: 'Successfully run. Total query runtime: 145 msec. 9 rows affected.'

Task 13. List the top 5 most recently created airlines.

Explanation: DESC puts the newest airlines first and LIMIT 5 returns five rows.

```
SELECT airline_id, airline_name, airline_country, created_at
FROM Airline
ORDER BY created_at DESC
LIMIT 5;
```



The screenshot shows the pgAdmin 4 interface with a SQL query executed in the 'Query' tab. The query is:

```
1 SELECT airline_id, airline_name, airline_country, created_at
2 FROM Airline
3 ORDER BY created_at DESC
4 LIMIT 5;
```

The 'Data Output' tab displays the results of the query in a table format. The table has four columns: `airline_id` (integer), `airline_name` (character varying (100)), `airline_country` (character varying (50)), and `created_at` (timestamp without time zone). The results show the top 5 most recently created airlines.

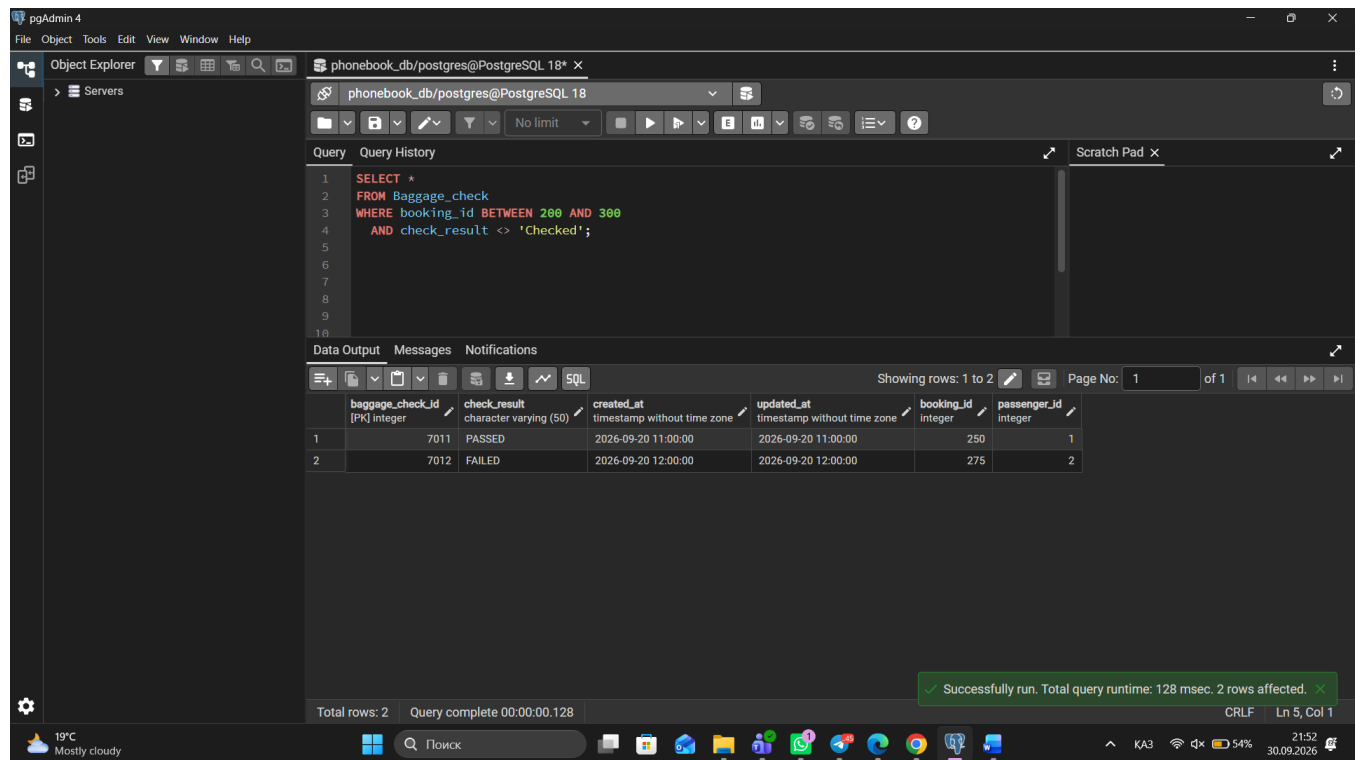
airline_id	airline_name	airline_country	created_at
1	Kazakhstan Airlines	Kazakhstan	2026-09-15 10:00:00
2	FlyStep	Kazakhstan	2026-09-14 10:00:00
3	SkyNomad	Kazakhstan	2026-09-13 10:00:00
4	Turkic Line	Turkey	2026-09-12 10:00:00
5	Asia Travel Air	Uzbekistan	2026-09-11 10:00:00

A green status bar at the bottom of the results area indicates: "Successfully run. Total query runtime: 143 msec. 5 rows affected."

Task 14. Return rows where booking_id is between 200 and 300 inclusive and check_result is not "Checked".

Explanation: BETWEEN includes 200 and 300. <> means not equal.

```
SELECT *
FROM Baggage_check
WHERE booking_id BETWEEN 200 AND 300
      AND check_result <> 'Checked';
```



The screenshot shows the pgAdmin 4 interface with a SQL query executed in the 'phonebook_db/postgres@PostgreSQL 18' database. The query is:

```
SELECT *
FROM Baggage_check
WHERE booking_id BETWEEN 200 AND 300
      AND check_result <> 'Checked';
```

The results are displayed in a table with the following columns: baggage_check_id [PK] integer, check_result character varying (50), created_at timestamp without time zone, updated_at timestamp without time zone, booking_id integer, and passenger_id integer. The table contains two rows:

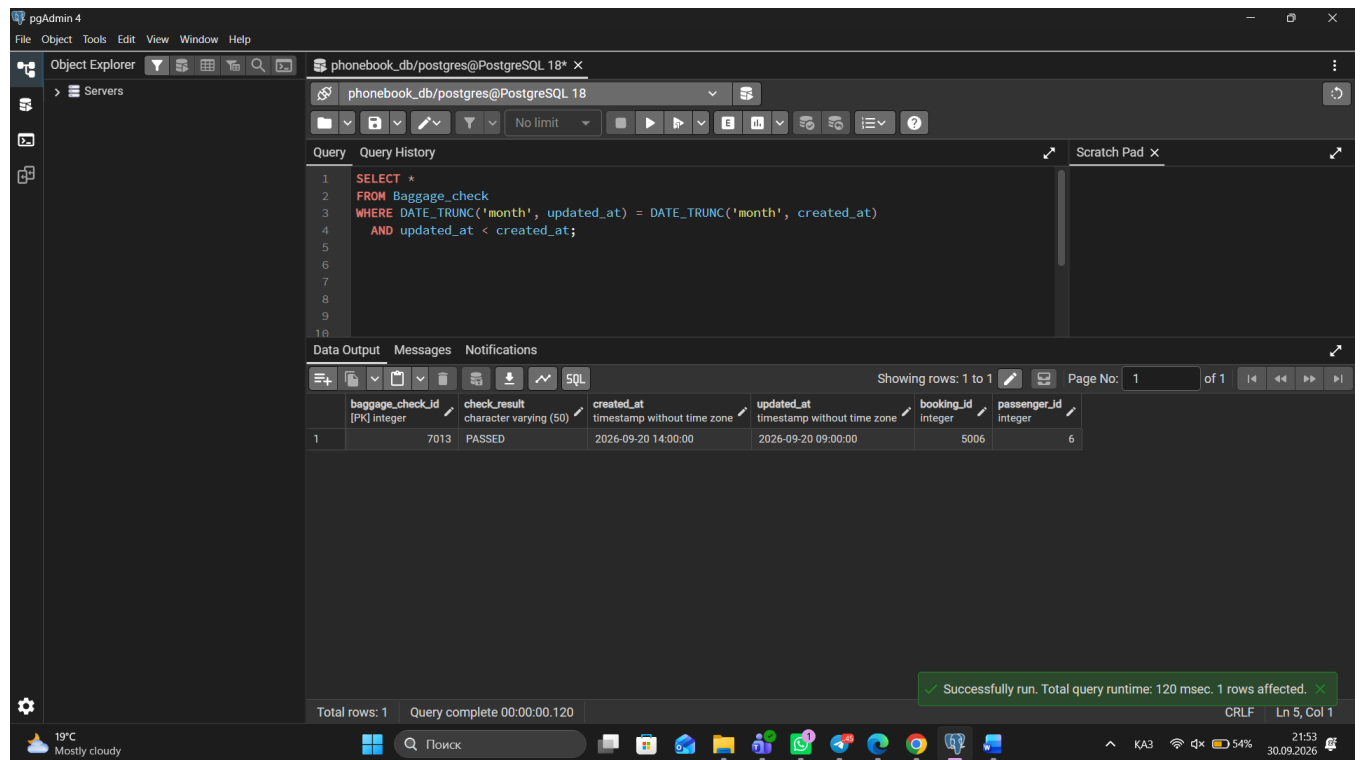
baggage_check_id [PK] integer	check_result character varying (50)	created_at timestamp without time zone	updated_at timestamp without time zone	booking_id integer	passenger_id integer
1	PASSED	2026-09-20 11:00:00	2026-09-20 11:00:00	250	1
2	FAILED	2026-09-20 12:00:00	2026-09-20 12:00:00	275	2

A status bar at the bottom indicates: Total rows: 2, Query complete 00:00:00.128, and a green message: Successfully run. Total query runtime: 128 msec. 2 rows affected. The system tray at the bottom shows the date and time as 21:52 30.09.2026.

Task 15. Baggage checks where `updated_at` is in the same month as `created_at` but earlier than `created_at`.

Explanation: `DATE_TRUNC` compares the month. The second condition checks that `updated_at` is earlier.

```
SELECT *
FROM Baggage_check
WHERE DATE_TRUNC('month', updated_at) = DATE_TRUNC('month', created_at)
      AND updated_at < created_at;
```



The screenshot shows the pgAdmin 4 interface with a SQL query executed in the 'Query' tab. The query is:

```
1 SELECT *
2 FROM Baggage_check
3 WHERE DATE_TRUNC('month', updated_at) = DATE_TRUNC('month', created_at)
4      AND updated_at < created_at;
```

The 'Data Output' tab shows the results of the query, displaying a single row with the following data:

baggage_check_id [PK] integer	check_result character varying (50)	created_at timestamp without time zone	updated_at timestamp without time zone	booking_id integer	passenger_id integer
1	7013 PASSED	2026-09-20 14:00:00	2026-09-20 09:00:00	5006	6

A status bar at the bottom indicates: 'Total rows: 1 Query complete 00:00:00.120' and a green message: 'Successfully run. Total query runtime: 120 msec. 1 rows affected.'